

Principia Spec Delta

COM Projection, Invariant Monitoring, and
Physics Visualisation Modes

Integration into the Implementation Spec

Malachy Doherty

March 2026

Abstract

This document specifies the exact changes required to integrate COM projection, invariant monitoring, and a comprehensive set of physics visualisation modes into the existing Principia v1 implementation spec. It covers: additions to the `SampleSummary` struct, modifications to the per-sample integration kernel, new uniform buffer fields, a full taxonomy of colour mapping modes for physical quantities (energy, angular momentum, mass ratios, momenta, Jacobi vectors, and derived diagnostics), and GUI surface changes. Each section identifies the insertion point in the existing spec by section number.

Contents

1	Overview of touched spec sections	3
2	SampleSummary struct changes	3
3	Uniform buffer additions	4
4	Integration kernel modifications	5
4.1	Initialisation (before the integration loop)	5
4.2	IC-derived physics quantities (before the integration loop)	6
4.3	COM projection (inside the integration loop)	8
4.4	Invariant accumulation (inside the integration loop)	9
4.5	Final write-out (after the integration loop)	9
5	Physics visualisation modes	9
5.1	Group 1: Conserved quantities and invariants	10
5.2	Group 2: Mass and configuration geometry	10
5.3	Group 3: Trajectory-derived quantities	11
5.4	Group 4: Numerical diagnostics	12
5.5	Compositing model	12
5.6	GUI integration	12
6	GUI diagnostic panel	13
6.1	Global health indicators	13
6.2	Per-pixel tooltip	13

7	Data export format changes	14
7.1	Binary header	14
7.2	Per-sample record	14
7.3	CSV / JSON export	15
8	Implementation plan integration	15
9	Validation	16
10	Spec summary	17

1 Overview of touched spec sections

Existing spec section	What changes	Nature of change
SampleSummary struct	Diagnostic + physics fields	Additive
Per-sample compute kernel	Projection step + invariant accumulation	Insert
Per-sample compute kernel	IC-derived physics field writes	Insert
Uniform buffer layout	Stabilising floors $\varepsilon_E, \varepsilon_L$	Additive
Colour mapping / overlays	Physics vis modes + drift overlays	Additive
Data export format	New fields in binary header	Additive
GUI diagnostic panel	Invariant health readout	Additive
GUI colour mode selector	Physics visualisation group	Additive

No existing fields are removed or reinterpreted. All changes are additive or insertions into the existing integration loop.

2 SampleSummary struct changes

Insertion point: the existing SampleSummary struct definition.

Add the following fields to the per-sample output struct:

```
// --- COM projection and invariant diagnostics ---

// Energy drift
f32 delta_E_final;    // E(t_end) - E(0)
f32 delta_E_max_abs;  // max_{tau <= t_end} |E(tau) - E(0)|

// Angular momentum drift
f32 delta_Lz_final;   // Lz(t_end) - Lz(0)
f32 delta_Lz_max_abs; // max_{tau <= t_end} |Lz(tau) - Lz(0)|

// Relative drift (for colour mapping)
f32 delta_E_rel;      // delta_E_final / max(eps_E, |E(0)|)
f32 delta_Lz_rel;     // delta_Lz_final / max(eps_L, |Lz(0)|)

// Initial invariant values (needed for relative drift
// computation and for downstream analysis)
f32 E_0;              // E(0)
f32 Lz_0;             // Lz(0)

// --- IC-derived physics quantities ---
// (computed once from the decoded initial condition,
// before the integration loop begins)

// Masses
f32 m1;               // mass of body 1
f32 m2;               // mass of body 2
f32 m3;               // mass of body 3

// Mass ratios (chart-dependent; only vary across the
// slice for charts that parameterise mass)
f32 mu_12;            // m1 / m2
```

```

f32 mu_13;           // m1 / m3
f32 q_mass;          // m_min / M_total (mass fraction
                      // of lightest body: 0 = test particle,
                      // 1/3 = equal mass)

// Jacobi vectors (from the decoded IC)
f32 rho1_mag;        // |rho_1|
f32 rho2_mag;        // |rho_2|
f32 rho_ratio;       // |rho_2| / |rho_1|
f32 rho_angle;       // angle between rho_1 and rho_2 [rad]

// Momenta
f32 K_0;             // total kinetic energy at t=0
f32 V_0;             // gravitational potential at t=0
f32 virial_ratio;     // 2*K_0 / |V_0| (= 1 for virial
                      // equilibrium)

// Pairwise geometry
f32 r_min_pair_0;     // min_{i<j} |r_i - r_j| at t=0
                      // (closest pair distance)
f32 r_min_pair_final; // min_{i<j} |r_i - r_j| over the
                      // whole trajectory (closest approach)

// Trajectory-derived
f32 escape_time;      // time to termination (escape,
                      // collision, or horizon)
u32 n_close;          // number of close encounters
                      // (r_ij < r_close threshold)

```

Sizing. The invariant diagnostics add eight `f32` fields (32 bytes). The IC-derived physics quantities add a further fifteen `f32` fields and one `u32` field (64 bytes). Total additional payload: 96 bytes per sample. For a 1024×1024 tile this is ~ 96 MB, which is modest relative to the existing `SampleSummary` payload including checkpoint arrays.

Bit-packing note. These fields are continuous `f32` values, not bit-packed flags. They sit alongside the existing bit-packed `descriptor` and `outcome` fields without interfering. If struct alignment requires it, pad to a 16-byte boundary after the last new field.

Design rationale: why both absolute and relative drift. Absolute drift (`delta.E_final`, `delta.Lz_final`) is the physically meaningful quantity. Relative drift (`delta.E_rel`, `delta.Lz_rel`) is useful for colour mapping because it normalises across regions of the IC manifold where $|E(0)|$ varies by orders of magnitude. The max-abs fields (`delta.E_max_abs`, `delta.Lz_max_abs`) catch transient close-encounter spikes that may recover by the end of the trajectory—exactly the regime where the Burrau dynamics are most interesting.

3 Uniform buffer additions

Insertion point: the existing per-dispatch uniform buffer layout.

Add to the simulation parameter uniform block:

```

// Stabilising floors for relative drift computation.
// Prevents division by near-zero when E(0) or Lz(0)
// is close to zero (e.g. parabolic energy, zero angular
// momentum rest configurations).
f32 eps_E;      // default: 1e-6
f32 eps_L;      // default: 1e-6

// Close-encounter threshold for n_close counting.
f32 r_close;    // default: 0.01 (normalised units)

```

These are set once at dispatch time by the CPU and do not change per-step. Default values are appropriate for the normalised unit system used by the atlas decoder. If the simulator later supports physical units, these should scale accordingly.

4 Integration kernel modifications

Insertion point: the per-sample integration loop, between the final integration step and the `SampleSummary` write-out.

4.1 Initialisation (before the integration loop)

After decoding the initial condition and before entering the timestep loop, evaluate and store the initial invariants:

```

// --- Invariant initialisation ---
let E_0: f32 = compute_energy(r, v, m); // Eq. (18) of mini spec
let Lz_0: f32 = compute_Lz(r, v, m);   // Eq. (19) of mini spec

// Running max-abs accumulators
var delta_E_max_abs: f32 = 0.0;
var delta_Lz_max_abs: f32 = 0.0;

```

Where `compute_energy` and `compute_Lz` are:

```

fn compute_energy(
  r: array<vec2<f32>, 3>,
  v: array<vec2<f32>, 3>,
  m: array<f32, 3>
) -> f32 {
  // Kinetic energy
  var T: f32 = 0.0;
  for (var i = 0u; i < 3u; i++) {
    T += 0.5 * m[i] * dot(v[i], v[i]);
  }

  // Gravitational potential (G = 1 in natural units)
  var V: f32 = 0.0;
  for (var i = 0u; i < 3u; i++) {
    for (var j = i + 1u; j < 3u; j++) {
      let dr = r[i] - r[j];
      let dist = length(dr);
      V -= m[i] * m[j] / dist;
    }
  }
}

```

```

    }
  }

  return T + V;
}

fn compute_Lz(
  r: array<vec2<f32>, 3>,
  v: array<vec2<f32>, 3>,
  m: array<f32, 3>
) -> f32 {
  var Lz: f32 = 0.0;
  for (var i = 0u; i < 3u; i++) {
    // 2D cross product: x * vy - y * vx
    Lz += m[i] * (r[i].x * v[i].y - r[i].y * v[i].x);
  }
  return Lz;
}

```

Note on force reuse. The pairwise distances $\|r_i - r_j\|$ and the mass products $m_i m_j$ computed here are the same quantities needed for the gravitational force evaluation. If the integrator caches these (which it should for the leapfrog/symplectic step), the potential energy evaluation is essentially free—just sum the cached $-m_i m_j / \|r_i - r_j\|$ terms. The kinetic energy term requires three dot products, which is negligible.

4.2 IC-derived physics quantities (before the integration loop)

Immediately after invariant initialisation, compute and store the per-sample physics quantities that derive from the decoded initial condition. These are evaluated once and written directly to `SampleSummary`; they do not change during integration.

```

// --- IC-derived physics quantities ---
// (r, v, m are the decoded initial state;
// rho1, rho2 are the Jacobi vectors from the decoder)

// Masses (store for downstream use; the decoder
// already has these)
summary.m1 = m[0];
summary.m2 = m[1];
summary.m3 = m[2];

// Mass ratios
summary.mu_12 = m[0] / m[1];
summary.mu_13 = m[0] / m[2];
let m_min = min(m[0], min(m[1], m[2]));
summary.q_mass = m_min / M_total;

// Jacobi vectors
// rho1 and rho2 come from the atlas decoder;
// if the integrator works in Cartesian coordinates,
// reconstruct from positions:
// rho1 = r[1] - r[0]

```

```

//    com12 = (m[0]*r[0] + m[1]*r[1]) / (m[0] + m[1])
//    rho2 = r[2] - com12
let rho1 = r[1] - r[0];
let com12 = (m[0] * r[0] + m[1] * r[1]) / (m[0] + m[1]);
let rho2 = r[2] - com12;

summary.rho1_mag = length(rho1);
summary.rho2_mag = length(rho2);
summary.rho_ratio = summary.rho2_mag
    / max(1e-30, summary.rho1_mag);
// Signed angle from rho1 to rho2
summary.rho_angle = atan2(
    rho1.x * rho2.y - rho1.y * rho2.x,
    dot(rho1, rho2)
);

// Energetics
// (E_0 already computed in invariant initialisation)
var K_0: f32 = 0.0;
for (var i = 0u; i < 3u; i++) {
    K_0 += 0.5 * m[i] * dot(v[i], v[i]);
}
summary.K_0 = K_0;
summary.V_0 = E_0 - K_0; // V = E - T
summary.virial_ratio = 2.0 * K_0
    / max(1e-30, abs(summary.V_0));

// Pairwise geometry at t=0
var r_min_0: f32 = 1e30;
for (var i = 0u; i < 3u; i++) {
    for (var j = i + 1u; j < 3u; j++) {
        let d = length(r[i] - r[j]);
        r_min_0 = min(r_min_0, d);
    }
}
summary.r_min_pair_0 = r_min_0;

// Trajectory-derived accumulators (initialise here,
// update during integration)
var r_min_pair_running: f32 = r_min_0;
var n_close: u32 = 0u;

```

Jacobi vector convention. The Jacobi decomposition pairs bodies 1–2 as the inner pair and body 3 as the outer, consistent with the atlas decoder convention. If the decoder uses a different pairing (e.g. for different chart families), the same formulae apply with the appropriate index permutation.

Close-encounter counting. The threshold r_{close} for incrementing `n_close` should be a uniform parameter (default: $r_{\text{close}} = 0.01$ in normalised units). Inside the integration loop, after each step:

```

// --- Close encounter tracking (inside loop) ---
for (var i = 0u; i < 3u; i++) {

```

```

    for (var j = i + 1u; j < 3u; j++) {
        let d = length(r[i] - r[j]);
        r_min_pair_running = min(r_min_pair_running, d);
        if (d < uniforms.r_close) {
            n_close += 1u;
        }
    }
}

```

And at trajectory termination:

```

summary.r_min_pair_final = r_min_pair_running;
summary.n_close = n_close;
summary.escape_time = t; // current simulation time

```

4.3 COM projection (inside the integration loop)

After each integration substep, insert:

```

// --- COM projection (every timestep) ---
// Position: remove COM drift
var R_com = vec2<f32>(0.0);
var M_total: f32 = 0.0;
for (var i = 0u; i < 3u; i++) {
    R_com += m[i] * r[i];
    M_total += m[i];
}
R_com /= M_total;

for (var i = 0u; i < 3u; i++) {
    r[i] -= R_com;
}

// Velocity: remove bulk momentum drift
var V_com = vec2<f32>(0.0);
for (var i = 0u; i < 3u; i++) {
    V_com += m[i] * v[i];
}
V_com /= M_total;

for (var i = 0u; i < 3u; i++) {
    v[i] -= V_com;
}

```

Integrator placement. For a leapfrog (Störmer–Verlet) integrator with the kick-drift-kick structure, the projection should occur after the full step is complete (i.e. after the second half-kick), not between the half-kick and the drift. This ensures the symplectic structure of the individual step is not broken. The projection itself is a canonical linear map, so it commutes with the symplectic structure in the exact-arithmetic limit; in `f32` the ordering matters only at the roundoff level, but placing it after the full step is the cleanest convention.

Total mass caching. M_{total} is constant across the trajectory. Compute it once during initialisation and pass it as a local variable rather than recomputing per step.

4.4 Invariant accumulation (inside the integration loop)

After the COM projection and before the next timestep, evaluate the current invariant drift and update the running maximum:

```
// --- Invariant accumulation (every timestep) ---
let E_now: f32 = compute_energy(r, v, m);
let Lz_now: f32 = compute_Lz(r, v, m);

let dE = abs(E_now - E_0);
let dLz = abs(Lz_now - Lz_0);

delta_E_max_abs = max(delta_E_max_abs, dE);
delta_Lz_max_abs = max(delta_Lz_max_abs, dLz);
```

Cost. One energy evaluation and one L_z evaluation per timestep. As noted above, the pairwise distances are already available from the force computation, so the marginal cost is roughly six multiplies, three additions, three dot products, and two comparisons per step. This is negligible relative to the force evaluation and integration arithmetic.

4.5 Final write-out (after the integration loop)

After the trajectory terminates (escape, collision, or horizon reached), compute the final drift values and write to `SampleSummary`:

```
// --- Final invariant diagnostics ---
let E_final = compute_energy(r, v, m);
let Lz_final = compute_Lz(r, v, m);

summary.E_0 = E_0;
summary.Lz_0 = Lz_0;

summary.delta_E_final = E_final - E_0;
summary.delta_Lz_final = Lz_final - Lz_0;

summary.delta_E_max_abs = delta_E_max_abs;
summary.delta_Lz_max_abs = delta_Lz_max_abs;

// Relative drift with stabilising floor
summary.delta_E_rel =
    summary.delta_E_final / max(uniforms.eps_E, abs(E_0));
summary.delta_Lz_rel =
    summary.delta_Lz_final / max(uniforms.eps_L, abs(Lz_0));
```

5 Physics visualisation modes

Insertion point: the existing colour mapping specification, alongside the physics overlay modes.

The existing colour mapping system supports basin colouring, shape sphere, and angular velocity differential modes. This section adds a comprehensive set of physics visualisation modes organised into four groups. Each mode maps a per-sample scalar from `SampleSummary` to a colour via a 1D palette lookup.

5.1 Group 1: Conserved quantities and invariants

These modes visualise the physical invariants of each initial condition. They reveal the energy and angular-momentum landscape of the IC manifold—structure that is invisible in the basin colouring but physically fundamental.

Mode name	Source field	Palette	Notes
ENERGY	E_0	Diverging (centred at $E = 0$)	Negative bound, positive unbound. Zero contour = parabolic.
ANG_MOMENTUM	Lz_0	Diverging (centred at $L_z = 0$)	Sign distinguishes prograde/retrograde.
KINETIC	K_0	Sequential	Always ≥ 0 .
POTENTIAL	V_0	Sequential (inverted)	Always ≤ 0 . More negative = deeper well.
VIRIAL	virial_ratio	Diverging (centred at 1)	= 1 at virial equilibrium. < 1 bound-dominated.

Energy colour scale. The energy range across a typical Burrau-family slice spans several orders of magnitude (deeply bound binaries to marginally unbound escapers). A symmetric log scale is recommended: linear near zero (within $[-\varepsilon, +\varepsilon]$), logarithmic outside. The zero-energy contour should be visually distinct (e.g. a thin white or black line) since it separates bound and unbound configurations.

Angular momentum colour scale. L_z is signed. A diverging palette centred at zero naturally distinguishes prograde ($L_z > 0$) from retrograde ($L_z < 0$) configurations. For the Burrau rest configurations $L_z = 0$ everywhere, so this mode becomes informative only when the chart parameterises initial momentum.

5.2 Group 2: Mass and configuration geometry

These modes visualise the shape and mass structure of the initial configuration. They are most informative on charts that vary mass or geometry across the slice (e.g. the mass-simplex chart, the Burrau ν -sweep).

Mode name	Source field	Palette	Notes
MASS_RATIO_12	mu_12	Diverging (centred at 1)	Centred at 1 (equal mass pair).
MASS_RATIO_13	mu_13	Diverging	Centred at 1.
MASS_FRACTION	q_mass	Sequential	0 = test particle, 1/3 = equal mass.
JACOBI_RH01	rho1_mag	Sequential	Inner pair separation.
JACOBI_RH02	rho2_mag	Sequential	Outer body distance.
JACOBI_RATIO	rho_ratio	Log-scale diverging (centred at 1)	$ \rho_2 / \rho_1 $. > 1: hierarchical. < 1: compact.
JACOBI_ANGLE	rho_angle	Cyclic	$[-\pi, \pi]$. Use a cyclic palette (e.g. HSL hue wheel).
MIN_PAIR_DIST	r_min_pair_0	Sequential (log scale)	Closest pair at $t = 0$. Small = near-collision IC.

Jacobi ratio interpretation. The ratio $|\rho_2|/|\rho_1|$ is a scale-free measure of how hierarchical the configuration is. Values $\gg 1$ indicate a tight inner pair with a distant third body (the hierarchical triple regime where perturbation theory works well). Values ~ 1 indicate a compact configuration where all three bodies interact comparably (the chaotic regime). This is arguably the single most physically informative scalar you can extract from the geometry.

Jacobi angle colour scale. Since the angle wraps at $\pm\pi$, the palette must be cyclic—the colour at $-\pi$ and $+\pi$ must match. Standard cyclic palettes (twilight, HSL hue, phase colourmaps) work here. This mode should integrate with the existing colour mapping system’s OKLAB pipeline; a cyclic VMF-OKLAB mode would be the natural implementation.

5.3 Group 3: Trajectory-derived quantities

These modes visualise properties of the integrated trajectory, not just the initial condition. They require the integration to have completed.

Mode name	Source field	Palette	Notes
ESCAPE_TIME	escape_time	Sequential (log scale)	Time to termination. Long-lived = fractal boundary.
CLOSE_ENCOUNTERS	n_close	Sequential (integer)	Count of close approaches. High = chaotic scattering.
MIN_APPROACH	r_min_pair_final	Sequential (log scale)	Closest approach over entire trajectory.

Escape time and fractal dimension. The escape time map is closely related to the fractal dimension of basin boundaries. Pixels with long escape times cluster on the boundaries between basins—they are the orbits that take longest to “decide” which outcome to reach. Rendering escape time in log scale with a perceptually uniform palette produces striking images where the fractal structure of the boundary is visible as luminance variation.

Close encounter count. This is an integer field. The palette should use discrete steps for small counts (0–10) and transition to a continuous gradient for large counts. Regions with high encounter counts correlate with chaotic scattering and long-lived triple interactions.

5.4 Group 4: Numerical diagnostics

These modes visualise the numerical fidelity of the integration, as specified in the COM projection mini spec.

Mode name	Source field	Palette	Notes
ENERGY_DRIFT_ABS	<code>delta_E_max_abs</code>	Log-scale	Absolute worst-case.
ENERGY_DRIFT_REL	<code>delta_E_rel</code>	Log-scale	Normalised by $ E(0) $.
LZ_DRIFT_ABS	<code>delta_Lz_max_abs</code>	Log-scale	Absolute worst-case.
LZ_DRIFT_REL	<code>delta_Lz_rel</code>	Log-scale	Normalised by $ L_z(0) $.

Diagnostic colour scale. A sequential log-scale palette:

- Floor at $\sim 10^{-7}$ (single-precision machine epsilon regime).
- Ceiling at $\sim 10^0$ (drift comparable to the invariant itself).
- Regions with drift $> 10^{-2}$ should be visually distinct (warm/hot colours) as these indicate numerically suspect trajectories.

5.5 Compositing model

All physics visualisation modes operate in two regimes:

Primary mode. The mode replaces the basin colouring entirely. The palette maps the scalar field to RGB directly. This is useful when the physics quantity itself is the object of study (e.g. “show me the energy landscape”).

Overlay mode. The mode is composited over the existing primary render (basin, shape sphere, etc.) as a semi-transparent layer. The alpha channel is derived from the scalar magnitude: regions where the quantity is near its midpoint or zero are transparent, regions where it is extreme are opaque. This is useful for “show me the basin map, but highlight where the energy is extreme” workflows.

The compositing pipeline should use premultiplied alpha and operate in linear OKLAB space to avoid hue shifts at intermediate opacities.

5.6 GUI integration

The colour mode selector should organise the new modes into named groups:

Group	Modes
Invariants	ENERGY, ANG_MOMENTUM, KINETIC, POTENTIAL, VIRIAL
Geometry	MASS_RATIO_12, MASS_RATIO_13, MASS_FRACTION, JACOBI_RHO1, JACOBI_RHO2, JACOBI_RATIO, JACOBI_ANGLE, MIN_PAIR_DIST
Trajectory	ESCAPE_TIME, CLOSE_ENCOUNTERS, MIN_APPROACH
Diagnostics	ENERGY_DRIFT_ABS, ENERGY_DRIFT_REL, LZ_DRIFT_ABS, LZ_DRIFT_REL

Each group is collapsible in the UI. The “Diagnostics” group should be selectable as an overlay independently of the other groups (i.e. you can have `ENERGY` as primary + `ENERGY_DRIFT_REL` as overlay simultaneously). The other groups are mutually exclusive with the existing primary modes (basin, shape sphere, angular velocity differential).

6 GUI diagnostic panel

Insertion point: the GUI spec, as a new panel or section of the existing debug/info panel.

6.1 Global health indicators

After a tile or frame completes rendering, the CPU-side reduction should compute and display:

Indicator	Computation
Worst energy drift	$\max_{\text{samples}} \text{delta_E_max_abs} $
Worst L_z drift	$\max_{\text{samples}} \text{delta_Lz_max_abs} $
Median energy drift	median of $ \text{delta_E_max_abs} $ over samples
Median L_z drift	median of $ \text{delta_Lz_max_abs} $ over samples
Suspect sample fraction	fraction with $ \text{delta_E_rel} > 10^{-2}$

Display. These belong in a collapsible “Numerical health” panel, visible by default in research mode, hidden by default in presentation mode. The worst-case values should have traffic-light colouring: green below 10^{-5} , amber between 10^{-5} and 10^{-2} , red above 10^{-2} .

6.2 Per-pixel tooltip

When the user hovers over a rendered pixel (if the cursor inspector is active), the tooltip should additionally display:

- $E(0)$, $L_z(0)$, $K(0)$, $V(0)$ for that sample.
- Mass triple (m_1, m_2, m_3) and mass fraction q .
- Jacobi magnitudes $|\rho_1|$, $|\rho_2|$, ratio, and angle.
- Virial ratio $2K/|V|$.
- Escape time and close-encounter count.
- ΔE_{max} and $\Delta L_{z,\text{max}}$ (the transient worst-case drift).
- δE_{rel} and δL_{rel} (the relative final drift).

This is a lot of data. The tooltip should be organised into collapsible groups matching the four colour mode groups (Invariants, Geometry, Trajectory, Diagnostics), with only the currently active group expanded by default.

7 Data export format changes

Insertion point: the binary export format specification (§26.2.4).

7.1 Binary header

Add to the export metadata header:

```
f32 eps_E;           // stabilising floor used
f32 eps_L;           // stabilising floor used
f32 r_close;         // close-encounter threshold used
u32 com_projection;  // 1 = projection active, 0 = disabled
u32 invariant_cadence; // steps between invariant evaluations
                      // (1 = every step, default)
```

7.2 Per-sample record

All 24 new `SampleSummary` fields are appended to each per-sample record in the binary export. The field order matches the struct definition in §2. Downstream consumers (e.g. `np.frombuffer`) need the updated struct dtype to parse correctly.

A recommended NumPy dtype fragment for the new fields:

```
physics_dtype = np.dtype([
    # Invariant diagnostics
    ('delta_E_final', 'f4'), ('delta_E_max_abs', 'f4'),
    ('delta_Lz_final', 'f4'), ('delta_Lz_max_abs', 'f4'),
    ('delta_E_rel', 'f4'), ('delta_Lz_rel', 'f4'),
    ('E_0', 'f4'), ('Lz_0', 'f4'),
    # Masses
    ('m1', 'f4'), ('m2', 'f4'), ('m3', 'f4'),
    # Mass ratios
    ('mu_12', 'f4'), ('mu_13', 'f4'), ('q_mass', 'f4'),
    # Jacobi vectors
    ('rho1_mag', 'f4'), ('rho2_mag', 'f4'),
    ('rho_ratio', 'f4'), ('rho_angle', 'f4'),
    # Energetics
    ('K_0', 'f4'), ('V_0', 'f4'), ('virial_ratio', 'f4'),
    # Pairwise geometry
    ('r_min_pair_0', 'f4'), ('r_min_pair_final', 'f4'),
    # Trajectory-derived
    ('escape_time', 'f4'), ('n_close', 'u4'),
])
```

7.3 CSV / JSON export

For text-format exports, add all new fields as columns/keys. Group them with a naming prefix for clarity:

```
# Invariant diagnostics
E_0, Lz_0, delta_E_final, delta_Lz_final,
delta_E_max_abs, delta_Lz_max_abs,
delta_E_rel, delta_Lz_rel

# Mass and geometry
m1, m2, m3, mu_12, mu_13, q_mass,
rho1_mag, rho2_mag, rho_ratio, rho_angle,
r_min_pair_0

# Energetics
K_0, V_0, virial_ratio

# Trajectory-derived
escape_time, n_close, r_min_pair_final
```

8 Implementation plan integration

Where this lands in the milestone-ordered task breakdown.

The COM projection, invariant monitoring, and physics visualisation work splits across existing milestones:

Task	Milestone	Size
<i>Compute pipeline (M1)</i>		
Add diagnostic fields to <code>SampleSummary</code>	M1	S
Add IC-derived physics fields to <code>SampleSummary</code>	M1	S
Add trajectory-derived fields to <code>SampleSummary</code>	M1	S
Implement <code>compute_energy</code> , <code>compute_Lz</code>	M1	S
Insert COM projection into integration loop	M1	S
Insert invariant accumulation into loop	M1	S
Compute IC-derived physics quantities after decode	M1	S
Close-encounter counting in integration loop	M1	S
Wire all new fields to <code>SampleSummary</code> write-out	M1	S
Add ε_E , ε_L , r_{close} to uniforms	M1	S
<i>Colour mapping (M3)</i>		
Implement Group 1 modes (invariants)	M3	M
Implement Group 2 modes (geometry)	M3	M
Implement Group 3 modes (trajectory)	M3	S
Implement Group 4 modes (diagnostics)	M3	S
Cyclic palette for Jacobi angle mode	M3	S
Symmetric log scale for energy mode	M3	S
Primary/overlay compositing pipeline	M3	M
Colour mode group selector in GUI	M3	S
<i>GUI (M4)</i>		
CPU-side reduction for global health indicators	M4	S
Numerical health panel in GUI	M4	S
Per-pixel tooltip: all physics fields, grouped	M4	M
<i>Export (M5)</i>		
Update binary export header and per-sample record	M5	S
Update CSV/JSON export with all new fields	M5	S

The M1 compute tasks should be done as a single unit of work—they are tightly coupled in the shader. The M3 colour mapping tasks are the largest block; Groups 1–2 are medium because they require new palette types (symmetric log, cyclic) and the compositing pipeline. Groups 3–4 reuse existing sequential palettes and are straightforward. The M4 and M5 tasks can be done independently whenever those milestones are reached.

9 Validation

To confirm the implementation is correct:

1. **COM conservation.** After projection, $\|R_{\text{com}}\|$ and $\|P_{\text{com}}\|$ should be at or below `f32` machine epsilon ($\sim 10^{-7}$) times the characteristic scale. If not, the projection arithmetic is wrong.
2. **Known orbit test.** For the Lagrange equilateral triangle solution (equal masses, circular orbit), E and L_z are exactly conserved analytically. The numerical drift should be bounded and oscillatory, not secularly growing. Run for $\sim 10^4$ steps and verify $|\Delta E|/|E(0)| < 10^{-4}$ (for a symplectic integrator in `f32` this is a reasonable expectation).

3. **Euler solution test.** For the collinear Euler solution, $L_z = 0$ exactly. Verify that $L_z(0) \approx 0$ and that δL uses the stabilising floor correctly without producing NaN or $\pm\infty$.
4. **Close encounter spike.** For a Burrau-family IC known to produce a close triple encounter, verify that `delta_E_max_abs` > `|delta_E_final|`—i.e. the transient spike is larger than the final drift, confirming the per-step accumulator is working.
5. **Jacobi vector consistency.** For a known configuration (e.g. the 3–4–5 Burrau triangle with unit hypotenuse), verify that the stored Jacobi magnitudes, ratio, and angle match the analytic values from the Euclid parametrisation formulae. The angle between ρ_1 and ρ_2 for the right-angle vertex at the origin should be exactly $\pi/2 + \arctan(a/b)$ (or its equivalent), which is straightforward to check.
6. **Virial ratio for rest configurations.** For any Burrau-family IC with zero initial velocity, $K(0) = 0$ and therefore the virial ratio should be exactly zero. Verify that `virial_ratio` = 0 (not NaN or Inf) when $K(0) = 0$, confirming the $\max(10^{-30}, |V(0)|)$ floor in the denominator works correctly.
7. **Close encounter count.** For the classic 3–4–5 Burrau problem, verify that `n_close` > 0 (it is known to produce close triple encounters). For a hierarchical triple with well-separated inner and outer orbits, verify `n_close` = 0 at reasonable threshold values.
8. **Export round-trip.** Export a tile in binary format, reload in Python with the updated dtype, and verify all fields (diagnostic and physics) are present, correctly aligned, and match the on-screen tooltip values.

10 Spec summary

The following additions are made to the Principia v1 implementation spec:

- **SampleSummary** gains 24 new fields (8 invariant diagnostics, 13 IC-derived physics quantities, 3 trajectory-derived quantities), totalling 96 bytes per sample.
- The uniform buffer gains two **f32** stabilising floors (ε_E , ε_L), one close-encounter threshold (r_{close}), and two **u32** configuration flags.
- The per-sample integration kernel gains: a COM projection step (after each full integration step), an invariant accumulation step, IC-derived physics computation (after decode, before integration), and close-encounter tracking (inside the integration loop).
- Two helper functions (`compute_energy`, `compute_Lz`) are added to the shader library.
- Twenty new colour visualisation modes are added, organised into four groups: Invariants (5), Geometry (8), Trajectory (3), and Diagnostics (4). These support both primary and overlay rendering with compositing in linear OKLAB space.
- The GUI gains a “Numerical health” panel, a grouped colour mode selector, and extended per-pixel tooltip with collapsible physics groups.
- The binary, CSV, and JSON export formats gain all new fields.

No existing behaviour is modified. The runtime cost is modest: COM projection adds ~ 20 FLOPs per step, the invariant evaluation reuses cached pairwise distances, and the IC-derived physics quantities are computed once per sample before integration begins. The close-encounter check adds three distance comparisons per step, which is negligible.